4-bit Unsigned Array Multiplier from RTL to GDSII on SkyWater SKY130 with Open-Source Tools

Sai Poojith
Roll No. 2025122010

Aneesh
Roll No. 2025122002

***Abstract*—We designed a 4-bit × 4-bit unsigned array multiplier in Verilog with registered inputs and outputs (16 flip-flops, 2-cycle latency), verified it exhaustively, synthesised it to the SKY130 `sky130_fd_sc_hd` library with Yosys, analysed its timing with OpenSTA and took it to a GDSII layout with LibreLane. RTL and gate-level simulation both pass all 256 input combinations. Synthesis gives 67 cells (784.502 μm^2) and STA gives $T_{\min} = 2.980$ ns ($F_{\max} = 335.57$ MHz) at the typical corner. LibreLane needed 7 runs and closed timing at a 7.0 ns clock across all its default corners, with 0 DRC and 0 LVS errors. Post-route $F_{\max}$ is 144.70 MHz at the worst (slow) corner and 288.04 MHz at the typical corner.**

I. Part A — RTL Design and Verification

A. Design

`rtl/full_adder.v` is a 1-bit full adder written only with `assign` statements: $sum = a \oplus b \oplus c_{in}$ and $c_{out} = ab + (a \oplus b)\, c_{in}$. The carry form shares the $a \oplus b$ term with the sum; it is logically identical to the majority form $ab + bc_{in} + ac_{in}$. `rtl/mult_array.v` registers `a` and `b` into `a_reg`/`b_reg` (8 flip-flops), forms the 16 partial-product bits $pp_i[j] = a[j] \wedge b[i]$, and adds them with three rows of four `full_adder` instances (`fa_r1_0` … `fa_r3_3`, 12 in total) wired exactly as in the connection table of Section 2 of the assignment. The 8-bit sum is registered into `p` (8 flip-flops). All 16 flip-flops are cleared by the asynchronous active-low reset `rst_n`. No `*` operator is used in the design. Fig. 1 shows the block diagram.

B. Testbench

`tb/tb_mult.v` generates a 10 ns clock, holds `rst_n` low for the first two cycles and applies all 256 combinations of a and b. Each input is applied on a falling edge; the testbench waits two rising edges (the 2-cycle latency) and compares `p` with `a*b` at the next falling edge. It prints a line for every mismatch, one summary line at the end, and dumps `mult.vcd`.

C. Results

RTL simulation:

```
PASS: 256 vectors checked, 0 errors
```

Error test (task 6), with `sum` changed to `a ^ b`:

```
MISMATCH: a=3 b=3 expected=9 got=5
...
FAIL: 256 vectors checked, 95 errors
```

After undoing the change the RTL simulation prints `PASS: 256 vectors checked, 0 errors` again.

D. Hand-worked example

Sai Poojith's roll number 2025122010 ends in 0, so $A = 0 + 5 = 5$ and $B = 7$.

```
        0 1 0 1   A = 5
      x 0 1 1 1   B = 7
---------------
        0 1 0 1   pp0 = A AND b0 (b0=1)
      0 1 0 1     pp1 = A AND b1 (b1=1), shifted 1
    0 1 0 1       pp2 = A AND b2 (b2=1), shifted 2
  0 0 0 0         pp3 = A AND b3 (b3=0), shifted 3
---------------
0 0 1 0 0 0 1 1   P = 35
```

Fig. 2 shows the same multiplication in GTKWave. The inputs $a = 5$, $b = 7$ are applied at 1760 ns, captured into the input registers at clock edge 1 (1765 ns), and the product $p = 35$ appears after clock edge 2 (1775 ns): a latency of 2 clock cycles.

II. Part B — Logic Synthesis

`synth/synth.tcl` runs the seven Yosys steps from the assignment (`read_verilog`, `synth -top mult_array -flatten`, `dfflibmap`, `abc`, `opt_clean -purge`, `stat`, `write_verilog`) with the `tt_025C_1v80` Liberty file. Table I is taken from `synth/mult_array_stat.rpt`.

TABLE I
Part B results (Yosys)

Metric	Value
Total number of cells	67
Number of flip-flops	16 (`dfrtp_1`)
Total cell area (μm^2)	784.502
Flip-flop area (μm^2)	$16 \times 25.024 = 400.384$
Flip-flop area (% of total)	51.04 %
3 most-used cell types	`dfrtp_1` (16), `nand2_1` (9), `xnor2_1` (7)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

The gate-level netlist was simulated with the assignment's command (`-DFUNCTIONAL -DUNIT_DELAY=#1`) and the same testbench, and passed. Our first gate-level run printed `FAIL: 256 vectors checked, 239 errors`. Every wrong value was the product of the previous vector. The testbench was sampling `p` 1 ns after the clock edge, which is exactly when the unit-delay flip-flop models update their

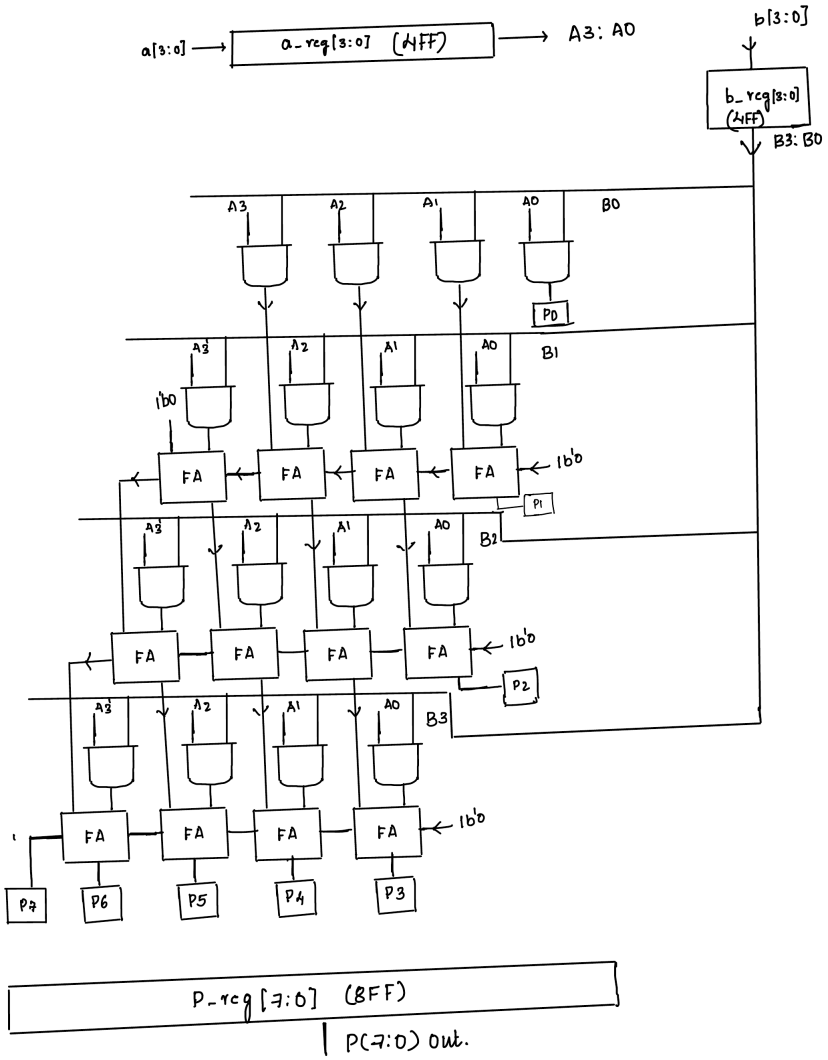


Fig. 1. Hand-drawn block diagram of mult_array: input registers a_reg and b_reg, the 16 AND gates (partial products), the 12 full adders in three rows with rippling carries, and the 8-bit output register p_reg.

outputs. We moved the comparison to the falling edge and reran all Part A and Part B simulations.

III. Part C — Static Timing Analysis

`sta/sta.tcl` runs the seven OpenSTA steps from the assignment at the typical corner. At $T = 10$ ns the worst setup slack is 7.020 ns, so

$$T_{\min} = 10 - 7.020 = 2.980 \text{ ns},$$
$$F_{\max} = 1000/2.980 = 335.57 \text{ MHz}.$$

Rerunning with $T = 2.98$ ns gives a slack of 0.000 ns (printed as `worst slack max 0.00`), which confirms $T_{\min}$.

The path starts at `b_reg[0]`, goes through the carry logic of the adder rows, and ends at the sum output of `fa_r3_3`, which is `p[6]`. There is no separate partial-product AND gate on this path. ABC merged the AND gates into the adder logic: `_048_` computes $a_3a_2b_0b_1 = pp_1[2]\,pp_0[3]$ in one cell, the carry-generate of `fa_r1_2`, whose two inputs are $pp_1[2]$ and $r_0[2] = pp_0[3]$. `_051_` also takes a_2 and b_0 directly. So all 8 logic gates (rows 2–9 of Table III) are full-adder carry

TABLE II
Part C results (OpenSTA, tt_025C_1v80)

Metric	Value
Worst slack at $T = 10$ ns (ns)	7.020
$T_{\min}$ (ns)	2.980 (rerun at 2.98)
$F_{\max}$ (MHz)	335.57
Slack at $T = T_{\min}$ (ns)	0.000
Critical path start point	`_106_` = `b_reg[0]` (`dfrtp_1`)
Critical path end point	`_100_` = `p_reg[6]` (`dfrtp_1`, D = `fa_r3_3.sum`)
Number of cells on the critical path	10 (launch FF + 8 gates + capture FF)
Latency (ns) $= 2 \times T_{\min}$	5.960
Throughput (M results/s)	335.57

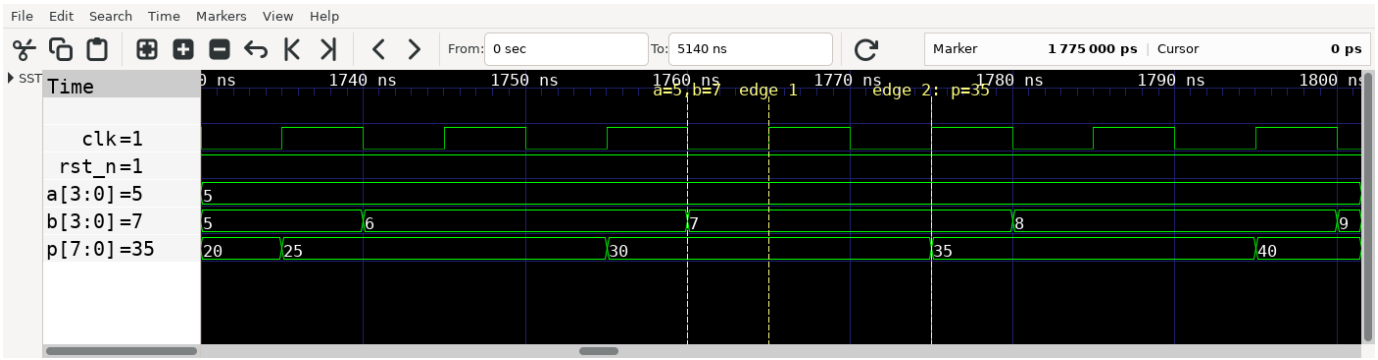


Fig. 2. GTKWave view of the 5×7 example (1730–1800 ns). Marker *a=5,b=7*: inputs applied (1760 ns); *edge 1*: inputs captured (1765 ns); *edge 2: p=35*: product registered (1775 ns).

TABLE III
CELLS ON THE CRITICAL PATH, IN ORDER (OPENSTA TIMING REPORT, $T = 2.98$ NS)

#	Instance	Cell	Delay (ns)	Arrival (ns)	Role
1	_106_	dfrtp_1	0.411	0.411	launch flip-flop b_reg[0] (CLK→Q)
2	_048_	and4_1	0.218	0.629	$a_3a_2b_0b_1 = pp_1[2]\,pp_0[3]$, carry-generate of fa_r1_2
3	_050_	a2111oi_0	0.428	1.057	adder carry logic
4	_051_	a21oi_1	0.094	1.151	adder carry logic (also takes $pp_0[2] = a_2b_0$)
5	_053_	mux2i_1	0.218	1.369	adder carry logic (inverting 2:1 mux)
6	_069_	xnor3_1	0.374	1.742	3-input XNOR (full-adder sum)
7	_074_	maj3_1	0.371	2.113	majority (full-adder carry)
8	_076_	maj3_1	0.376	2.489	majority (full-adder carry)
9	_080_	xor3_1	0.355	2.844	3-input XOR = fa_r3_3.sum
10	_100_	dfrtp_1	setup 0.136	—	capture flip-flop p_reg[6]

and sum logic, the same as the estimate of about $2N = 8$ full-adder delays in Section 2. The last four cells (xnor3_1, maj3_1, maj3_1, xor3_1) are direct full-adder sum and carry functions.

IV. PART D — PHYSICAL DESIGN

pnr/config.json contains exactly the six settings from the assignment: DESIGN_NAME, VERILOG_FILES, CLOCK_PORT, CLOCK_PERIOD, FP_SIZING = "absolute" and DIE_AREA = [0, 0, 150, 150]. The flow was run with librelane config.json (LibreLane v3.1.0.dev3) and its default timing corners: typical, slow and fast process corners (tt_025C_1v80, ss_100C_1v60, ff_n40C_1v95), each with nominal, minimum and maximum wire parasitics.

A. Timing closure

The starting period was $1.2\times T_{\min} = 1.2\times 2.980 = 3.576$ ns, rounded up to 4.0 ns. Every failing run failed only on setup timing in the slow corner ss_100C_1v60, where cells are much slower than at the typical corner used in Part C. Following the assignment, we increased CLOCK_PERIOD by 0.5 ns after each failure (Table IV). Synthesis was identical in all seven runs (68 cells, 765.734 μm^2), and so was the layout up to clock-tree synthesis (1252.45 μm^2 of standard cells). The runs differ only in the post-CTS timing-repair step (ResizerTimingPostCTS), which upsizes or buffers gates only where it predicts a setup violation from pre-route wire estimates:

- from 4.0 to 5.5 ns it added 197.69 μm^2, and the slack improved by about 0.5 ns per run;
- at 6.0 ns it added 180.17 μm^2;
- at 6.5 ns it added 23.77 μm^2;
- at 7.0 ns it added 12.51 μm^2.

With less repair, the slow-corner path got slower: data arrival was 5.97 ns up to 5.5 ns, then 6.15 ns at 6.0 ns, 6.66 ns at 6.5 ns and 6.91 ns at 7.0 ns. So the slack stopped improving at 6.0 ns, got worse from 6.0 to 6.5 ns ($-0.1282 \rightarrow -0.1568$ ns), and became positive at 7.0 ns ($+0.0890$ ns).

TABLE IV
LIBRELANE RUNS (FROM EACH RUN'S FINAL METRICS)

#	Period (ns)	Result	Worst slack (ns), ss	Viol.	tt slack (ns)
1	4.0	FAIL	−1.9527	15	0.8828
2	4.5	FAIL	−1.4527	12	1.3828
3	5.0	FAIL	−0.9527	9	1.8828
4	5.5	FAIL	−0.4563	6	2.3823
5	6.0	FAIL	−0.1282	3	2.8179
6	6.5	FAIL	−0.1568	3	3.0887
7	7.0	PASS	+0.0890	0	3.5283

Worst corner in every run: max_ss_100C_1v60. tt = nom_tt_025C_1v80. All runs: 0 DRC, 0 LVS.

B. Results

Post-route $F_{\max} = 1000/(\text{CLOCK_PERIOD} - \text{setup slack})$. We give it both for the worst corner, which is what LibreLane signs off on, and for the typical corner, which is the fair comparison with Part C:

$$\text{worst (ss): } 1000/(7.0 - 0.0890) = 144.70 \text{ MHz},$$
$$\text{typical (tt): } 1000/(7.0 - 3.5283) = 288.04 \text{ MHz}.$$

The 1901 instances are 16 flip-flops and 52 logic cells from LibreLane's own synthesis, plus 1833 cells added by the flow (Table V). LibreLane runs its own Yosys script, so its logic (68 cells, 765.734 μm^2) is not identical to our Part B netlist (67 cells). Hold timing also passes in every corner (worst hold slack 0.140 ns). Figs. 3 and 4 show the final layout.

V. Answers to Questions

Q1. Logic cells. Synthesis produced $67 - 16 = 51$ logic cells, not $16 + 12 = 28$, because the RTL's AND gates and full adders do not map one-to-one onto library cells. The design was flattened, so ABC optimised all the logic together. It dropped constant inputs (each row's carry-in and $r_0[3]$ are 0) and merged AND gates into adder logic (`and4_1` computes $a_3a_2b_0b_1$ directly). It also skipped the library's multi-output `fa_1` cell; its log says it ignored 9 multi-output cells. So each full adder becomes several single-output cells, such as an XOR3 for the sum plus a MAJ3 or AND-OR cells for the carry. How the RTL is written also matters. With the earlier majority form $c_{out} = ab + bc_{in} + ac_{in}$, the same flow gave 73 cells, 805.773 μm^2 and one `maj3`. The shared-$(a \oplus b)$ form gives 67 cells, 784.502 μm^2 and five `maj3`, even though both forms are logically identical. That version is smaller but slower ($T_{\min}$ 2.980 ns against 2.596 ns), because ABC's result depends on the structure it starts from.

Q2. Critical path. The critical path goes from input register `b_reg[0]` (`_106_`) to output register `p_reg[6]` (`_100_`, the sum of `fa_r3_3`). It has 10 cells: the launch flip-flop, 8 logic gates and the capture flip-flop. The logic part takes $2.844 - 0.411 = 2.433$ ns. All 8 gates are full-adder carry and sum logic, rippling through the rows to `fa_r3_3`. The partial-product AND gates are merged into them (`and4_1` computes $pp_1[2]\,pp_0[3]$, the carry-generate of `fa_r1_2`). That matches the $2N = 8$ full-adder estimate. The path ends with four direct full-adder cells (`xnor3`, `maj3`, `maj3`, `xor3`). The earlier gates (`and4`, `a2111oi`, `a21oi`, `mux2i`) are complex cells that combine carry terms from several adders.

Q3. Flip-flop area. The flip-flops take 400.384 of 784.502 μm^2, which is 51.04 %. Each `dfrtp_1` is 25.024 μm^2, because it contains two latches plus asynchronous reset logic. Most logic gates in our netlist are only 3.75–10.01 μm^2; just the 3 `xnor3`/`xor3` cells are close to a flip-flop in size (22.5–23.8 μm^2). The design is tiny (51 logic cells), so the 16 registers are a large share. For an N-bit multiplier the logic grows like N^2 but there are only $4N$ flip-flops, so in a bigger multiplier this percentage would drop.

Q4. Why post-route $F_{\max}$ is lower, and the added cells. Two effects lower it. The first is *multi-corner signoff*. Part C used only the typical corner, but LibreLane signs off on its slowest corner, `ss_100C_1v60`. There, on the critical path from `b_reg[1]` (`_109_`) to `p_reg[7]` (`_103_`, D = `fa_r3_3.cout`), data arrives at 6.915 ns instead of 3.416 ns at tt in the same layout. That gives the worst-corner $F_{\max}$ of 144.70 MHz. The second is *the real layout*: even at tt, $F_{\max}$ falls from 335.57 to 288.04 MHz. The tt effective period of 3.472 ns is the 3.097 ns data path (launch clock to D) plus a 0.124 ns setup time, LibreLane's 0.25 ns clock uncertainty and 0.0005 ns of net clock skew. The capture clock arrives 0.031 ns earlier than the launch clock (0.288 vs. 0.319 ns), and a 0.030 ns reconvergence credit almost cancels that. The clock-tree latency itself cancels because it delays both flip-flops. The data path is longer than Part C's 2.844 ns for three reasons. Routed wires add RC, so the launch flip-flop's clk→Q, with a fanout of 8, is 0.508 ns instead of 0.411 ns. LibreLane's own synthesis gives a different 68-cell netlist, whose critical path ends at `p[7]` instead of `p[6]`. And at 7.0 ns the post-CTS timing repair added only 12.51 μm^2. The flow also added cells: 245 tap cells tie the n-wells and substrate to VPWR/VGND to prevent latch-up, and 1305 decap cells are on-chip capacitors that steady the supply when many gates switch. 260 fill cells close the gaps in the rows so wells, implants and power rails stay continuous, and 3 `clkbuf_16` cells form the clock tree that reaches all 16 flip-flops with low skew. The remaining buffers are 17 `buf_2` on the input and output ports, 2 fanout buffers that split `rst_n` (16 reset pins exceed the maximum fanout of 10), and 1 `buf_4` inserted by the setup repair.

Q5. DRC and LVS. DRC (design rule check) verifies that the layout geometry obeys the foundry's manufacturing rules: minimum widths and spacings of each layer, enclosures, via sizes, densities and so on. LVS (layout versus schematic) extracts the transistors and connections from the layout and checks that they match the netlist, so the layout is the circuit we designed. Both must pass because they catch different failures. A DRC-clean layout can still be the wrong circuit, and a layout that matches the schematic can still be impossible to manufacture reliably (shorts, opens, low yield). Our final layout has 0 DRC errors (Magic and KLayout) and 0 LVS errors.

Q6. Scaling to 8 bits. From Section 2, an N-bit array multiplier uses N^2 AND gates and $N(N-1)$ full adders, and its longest path is about $2N$ full-adder delays.

$$N = 4: \quad 4 \cdot 3 = 12 \text{ FAs}, \quad 2 \cdot 4 = 8 \text{ FA delays}$$
$$N = 8: \quad 8 \cdot 7 = 56 \text{ FAs}, \quad 2 \cdot 8 = 16 \text{ FA delays}$$

So the number of full adders grows by $56/12 \approx 4.7\times$ (and the AND gates from 16 to 64), but the critical path only doubles. Scaling our Part C logic delay: $T_{\min} \approx 0.411 + 2 \times 2.433 + 0.136 \approx 5.41$ ns, so $F_{\max}$ would drop from about 336 MHz to about 185 MHz at the typical corner. Here 0.411 ns is the clk→Q delay, 2.433 ns is the logic delay and 0.136 ns is the setup time. The area grows roughly quadratically and the delay roughly linearly.

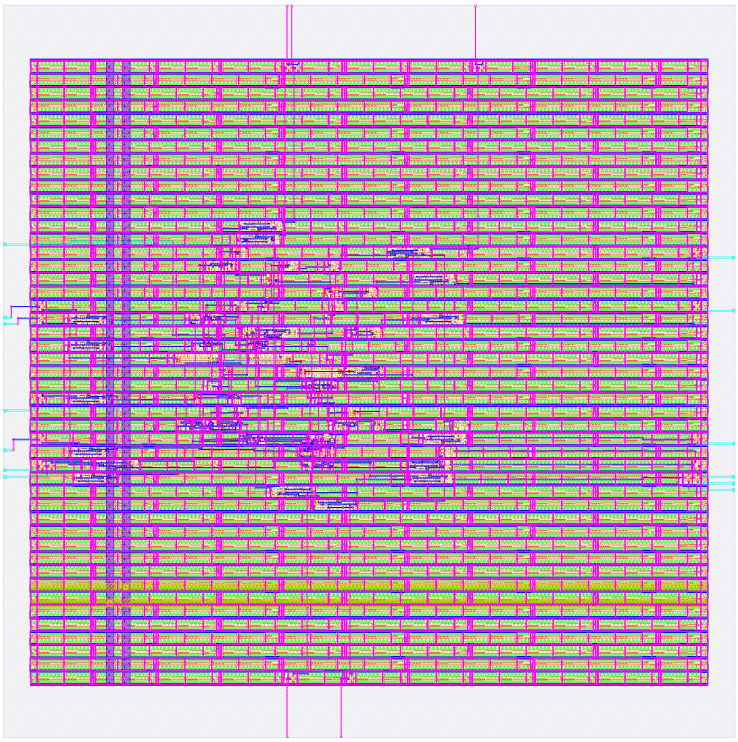


Fig. 3. Whole $150 \times 150\,\mu$m die (KLayout, `pnr/mult_array.gds`). The I/O pins sit on the die edge. The multiplier logic is clustered near the centre, and the rest of the core rows are filled with decap, fill and tap cells under the power grid.

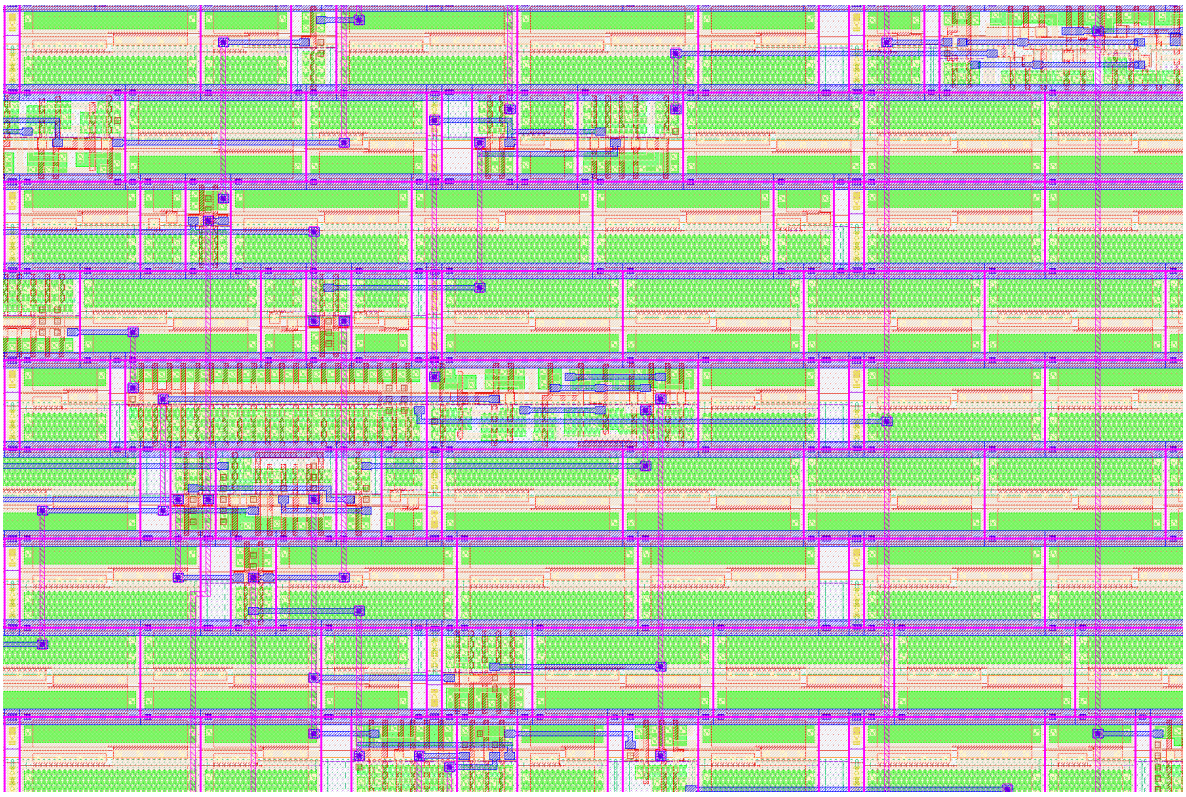


Fig. 4. Zoomed $36 \times 24\,\mu$m view at the die centre (colours from `sky130A.lyp`). Magenta outlines are the standard-cell boundaries (areaid layer); the cells sit in horizontal rows with blue metal-1 power rails along the row edges. Inside the cells, red is poly (transistor gates), green is diffusion and pale beige is li1, the local-interconnect layer that does most of the in-cell wiring ($681\,\mu\text{m}^2$ of li1 against $176\,\mu\text{m}^2$ of metal-1 in this window). Blue metal-1, a few magenta metal-2 wires and violet vias connect the cells.

TABLE V
PART D RESULTS (LIBRELANE FINAL RUN, 7.0 NS CLOCK)

Metric	Value
CLOCK_PERIOD used (ns)	7.0
Number of runs needed	7 (4.0, 4.5, 5.0, 5.5, 6.0, 6.5, 7.0 ns)
Core area (μm^2)	17 759.5 (die area 22 500)
Cells after synthesis (Part B)	67
Cells after place and route	1901 (336 standard cells + 1565 fill/decap)
Kinds of cells added	decap 1305 (`decap_12` 900, `decap_6` 193, `decap_3` 125, `decap_8` 50, `decap_4` 37); fill 260 (`fill_1` 218, `fill_2` 42); tap 245 (`tapvpwrvgnd_1`); buffers 23: 17 I/O port buffers (`buf_2`), 3 clock-tree buffers (`clkbuf_16`), 2 fanout buffers (`clkdlybuf4s50_1`) that split the `rst_n` net (16 reset pins, above the maximum fanout of 10), and 1 setup-repair buffer (`buf_4`)
Wirelength (μm)	2348
Setup slack (ns)	worst corner (`max_ss_100C_1v60`): 0.0890; typical (`nom_tt_025C_1v80`): 3.5283
Post-route $F_{\max}$ (MHz)	worst corner: 144.70; typical corner: 288.04
$F_{\max}$ from Part C (MHz)	335.57 (typical corner, no wires)
DRC errors	0 (Magic 0, KLayout 0)
LVS errors	0

VI. CONCLUSION

The multiplier passed exhaustive verification of all 256 input combinations in both RTL and gate-level simulation. Synthesis mapped it to 67 cells (784.502 μm^2), and STA at the typical corner gave $T_{\min} = 2.980$ ns. LibreLane closed timing at a 7.0 ns clock across all of its default corners, with 0 DRC and 0 LVS errors. Writing the full adder's carry with the shared $(a \oplus b)$ term gave a smaller but slower netlist than the majority form (784.502 vs. 805.773 μm^2, but 2.980 vs. 2.596 ns), so synthesis results depend on how the RTL is written. Routing, the clock tree and multi-corner signoff lowered post-route $F_{\max}$ from 335.57 MHz to 288.04 MHz at the typical corner and 144.70 MHz at the worst corner.

CONTRIBUTIONS

Both authors worked on every part of the assignment together and are familiar with the complete design and flow.

CODE AVAILABILITY

All source files, scripts and results are available at https://github.com/poojith06/4-bit_Array_Multiplier_from_RTL_to_GDSII.

REFERENCES

[1] N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Pearson, 2011, ch. 11.

[2] J. Rabaey, A. Chandrakasan, and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed. Pearson, 2003.

[3] S. Williams, "Icarus Verilog." [Online]. Available: https://steveicarus.github.io/iverilog

[4] "GTKWave." [Online]. Available: https://gtkwave.github.io/gtkwave

[5] YosysHQ, "Yosys Open SYnthesis Suite documentation." [Online]. Available: https://yosyshq.readthedocs.io/projects/yosys

[6] Parallax Software, "OpenSTA," manual `doc/OpenSTA.pdf`. [Online]. Available: https://github.com/parallaxsw/OpenSTA

[7] LibreLane contributors, "LibreLane." [Online]. Available: https://github.com/librelane/librelane

[8] M. Köfferlein, "KLayout documentation." [Online]. Available: https://klayout.de/doc.html

[9] SkyWater Technology and Google, "SkyWater SKY130 PDK documentation." [Online]. Available: https://skywater-pdk.readthedocs.io

[10] H. Pretl and G. Zachl, "IIC-OSIC-TOOLS." [Online]. Available: https://github.com/iic-jku/IIC-OSIC-TOOLS